

Questionari de preparació

Visualització de xarxes en 3D

Mario Vilar

16 de juny de 2025

Preguntes de connexió amb l'assignatura

X1. Com es relaciona el vostre projecte amb els conceptes generals de visualització de dades tractats a classe? El nostre projecte s'emmarca completament dins del model d'anàlisi de visualització de dades de Tamara Munzner que hem estudiat a classe, basat en els nivells de domini, abstracció, idioma i algorisme. L'abordem seguint el paradigma "What-Why-How":

- **What (Abstracció de dades):** Treballem amb un tipus de dades de **xarxa (network)**, on els nodes i les arestes poden tenir atributs quantitatius (com la centralitat) o categòrics (com la pertinença a una comunitat).
- **Why (Abstracció de tasques):** L'objectiu principal de l'usuari és **analitzar** per **descobrir** patrons. Concretament, vol explorar la **topologia** de la xarxa, els **camins** i identificar **característiques (features)** com ara clústers o valors extrems (nodes molt connectats).
- **How (Idioma visual):** Dissenyem un idioma visual basat en diagrames de node-enllaç. Usem **marks** de tipus punt (per als nodes) i línia (per a les arestes). Codifiquem informació mitjançant **canals visuals**: la posició espacial és el canal principal per mostrar l'estructura topològica, però podríem usar la mida per a un atribut quantitatiu o el color (hue) per a un de categòric, seguint el rànquing d'efectivitat dels canals vist a classe. Tècniques com l'Edge Bundling són una forma de **reduir** dades (agregant arestes) per millorar la claredat.

Pel que fa a l'**algorisme**, fem servir diferents implementacions de **Force-Directed Layout (FDL)** com a mètode per calcular la posició espacial dels nodes, que és el canal visual central. Aquest càlcul forma part del "How" en el sentit de Munzner, ja que determina com convertim l'estructura abstracta del graf en una representació visual interpretable:

- L'algorisme d'**Eades** introdueix el paradigma físic bàsic (forces atractives i repulsives), però té limitacions d'escalabilitat.
- **Fruchterman-Reingold** afegeix el concepte de **temperatura** per controlar el moviment i evitar mínims locals, millorant la convergència i l'estètica del layout.
- **Kamada-Kawai** utilitza distàncies de camí per ajustar les posicions, ideal per a xarxes petites on la **proporcionalitat de distàncies** és clau per la lectura de patrons globals.

- **Tutte** garanteix layouts plans i convexos resolent un sistema d'equacions; és rellevant com a base teòrica però poc pràctic per xarxes complexes o 3D.
- Els mètodes **multi-nivell** milloren el rendiment i la qualitat per a xarxes grans, i s'alineen amb el **principi d'escala** (scale-aware visualization) vist a classe.

Aquestes implementacions no només són algorismes, sinó que **defineixen directament el canal visual de posició** — i per tant, tenen un impacte crític en com l'usuari percep estructures com clústers, hubs o ponts. Això connecta amb els objectius del "Why": facilitar la descoberta visual de patrons estructurals.

X2. Com es compara la vostra aproximació 3D amb els exemples de visualització 2D que hem vist durant el curs? A l'assignatura s'ha destacat que la visualització 3D s'ha d'utilitzar amb precaució, ja que introdueix reptes importants com l'oclusió, els errors de perspectiva i la dificultat de navegació. Generalment, es reserva per a dades inherentment espacials, com en la Visualització Científica.

El nostre projecte pren un problema d'Informació Visual (una xarxa abstracta, sense geometria intrínseca) i el porta al 3D. Aquesta és una decisió de disseny conscient: no ho fem perquè les dades siguin 3D, sinó com una estratègia per guanyar espai de representació i intentar resoldre la congestió visual (l'efecte "hairball") que sovint pateixen els grafs densos en 2D. Aquesta decisió implica un compromís: a canvi de més espai per al layout, heretem tots els problemes perceptius del 3D. Per això, a diferència de moltes visualitzacions 2D estàtiques, la nostra aproximació exigeix una **interacció constant** per part de l'usuari (rotar, fer zoom) per poder ser entesa, seguint la mànta de Shneiderman "Overview first, Zoom and Filter, Details on Demand".

X3. Quins elements del pipeline gràfic heu posat en pràctica? Com es relaciona això amb els continguts de la part de gràfics? El nostre projecte implementa el **pipeline gràfic programable** modern que hem estudiat a fons, utilitzant la GPU per al renderitzat. El procés segueix el model "Forward-Mapping":

- CPU:** S'executa l'algorisme FDL per calcular les posicions 3D dels vèrtexs. Aquestes dades (posicions, normals, colors) es transfereixen a la memòria de la GPU (VRAM) mitjançant *Vertex Buffer Objects* (VBOs).
- GPU:** Un cop les dades són a la GPU, s'executa el pipeline:
 - **Vertex Shader:** Aquesta unitat programable s'executa per a cada vèrtex. La seva tasca principal és aplicar les transformacions de model-vista i de projecció (rebudes com a variables **uniform**) per calcular la posició final del vèrtex en coordenades de clip (**gl_Position**). En un esquema de Gouraud Shading, també calcularia aquí el color del vèrtex.
 - **Rasterització:** Les primitives (triangles) es converteixen en fragments.
 - **Fragment Shader:** S'executa per a cada fragment per determinar-ne el color final. Aquí es realitzen els càlculs d'il·luminació per píxel (Phong Shading) per obtenir un resultat visual de més qualitat, o s'utilitza el color interpolat que arriba del Vertex Shader.

- **Test de Visibilitat:** Finalment, s'utilitza el Z-Buffer per gestionar l'oclusió i escriure el color correcte al Frame Buffer.

Aquest flux de treball correspon exactament a l'arquitectura del pipeline programable d'OpenGL/GLSL que hem vist al Tema 3.

X4. Quines tècniques de representació gràfica de xarxes us han semblat més efectives en el vostre cas, i per què? En termes de la terminologia de l'assignatura, hem avaluat diferents idiomes visuals per a la representació de xarxes. Les dues famílies de tècniques més efectives que hem estudiat són:

- **Diagrames de node-enllaç amb posicionament per forces (Force-Directed Layouts):** Aquest és l'idioma base i el més efectiu. Utilitza la **posició espacial**, el canal visual de major efectivitat per a dades quantitatives, per codificar l'estructura topològica de la xarxa. En col·locar els nodes connectats més a prop, fa que les relacions i les comunitats siguin perceptibles de manera intuïtiva.
- **Agrupació d'arestes (Edge Bundling):** Aquesta tècnica és un complement molt eficaç que modifica l'idioma base per solucionar el seu principal inconvenient: la congestió visual en grafs densos. L'agrupació funciona com un mètode de **reducció** de dades (en aquest cas, de la complexitat visual), agregant arestes per revelar patrons de connexió a gran escala que altrament quedarien ocults.

Per tant, la combinació d'un layout per forces com a base, complementat amb l'agrupació d'arestes per a grafs densos, ens ha semblat l'aproximació més robusta i efectiva.

Preguntes generals sobre el projecte

- G1. Quin problema resol el vostre projecte?** El nostre projecte aborda el repte de **representar xarxes complexes en espais tridimensionals** de manera que les visualitzacions siguin clares, interpretables i útils. El problema principal és doble: per una banda, disposar els nodes i les arestes de manera que es preservin fidelment les relacions topològiques de les dades; per l'altra, mitigar la saturació visual i l'oclusió inherents als grafs densos en 3D.
- G2. Per què heu escollit la representació 3D en comptes de 2D?** Vam escollir la representació 3D principalment perquè **ofereix més espai** per a la disposició dels elements del graf. En visualitzacions 2D de xarxes complexes, és comú patir una gran congestió visual. L'eix addicional del 3D proporciona un l'lenç més gran que ofereix el potencial de "desembolicar" estructures complexes i reduir la superposició d'arestes, fent que la topologia de la xarxa sigui més fàcil de percebre.
- G3. Quins són els avantatges i inconvenients d'utilitzar xarxes 3D per visualitzar dades?**
- **Avantatges:**
 - **Més espai:** Permet una millor distribució dels nodes i pot reduir la congestió visual en comparació amb el 2D.

- **Revelació d'estructures:** Pot ajudar a descobrir patrons estructurals que no són evidents en un pla.
- **Immersió:** Constitueix una base natural per a futures visualitzacions en entorns de realitat virtual o augmentada (VR/AR).

- **Inconvenients:**

- **Oclusió:** Els nodes i les arestes es poden tancar entre si, dificultant la visió completa de l'estructura.
- **Navegació complexa:** Moure's i orientar-se en un espai 3D amb controls 2D (ratolí i teclat) és menys intuïtiu.
- **Distorsió de perspectiva:** La percepció de la mida i la distància pot ser enganyosa.
- **Llegibilitat del text:** Posicionar etiquetes de manera que siguin sempre llegibles és un repte significatiu.

G4. Com es gestionen els problemes d'oclusió o saturació visual en espais 3D? Gestionem aquests problemes mitjançant una combinació de tècniques algorísmiques, de renderitzat i d'interacció:

- **Edge Bundling:** És la nostra principal estratègia algorísmica per combatre la saturació. Agrupa les arestes que segueixen trajectòries similars en "feixos", reduint dràsticament l'efecte "hairball" fent més clara l'estructura general de connexions.
- **Interacció de l'usuari:** És un requisit fonamental. Permetre a l'usuari rotar, fer zoom i desplaçar la càmera li dona el poder de canviar el punt de vista per resoldre oclusions temporals i explorar la xarxa des de diferents angles.
- **Z-buffering:** A nivell de renderitzat, el pipeline gràfic utilitza el Z-Buffer (test de visibilitat) per assegurar que es dibuixen els objectes correctes, gestionant l'oclusió a nivell de maquinari.

G5. Quines tècniques heu utilitzat per millorar la llegibilitat de la visualització? Les dues famílies de tècniques principals que hem explorat, **Force-Directed Layouts (FDL)** i **Edge Bundling (EB)**, tenen com a objectiu millorar la llegibilitat:

- Els **FDL** milloren la llegibilitat organitzant els nodes de manera intuïtiva. L'objectiu és aconseguir una distribució uniforme dels nodes, minimitzar la congestió i mantenir longituds d'aresta consistents, la qual cosa fa que l'estructura (com els clústers) sigui visualment evident.
- L'**Edge Bundling** ataca directament la il·legibilitat causada per l'excés d'arestes en grafs densos. En agrupar-les, redueix el desordre visual i permet seguir els fluxos i les connexions principals a través de la xarxa.

A més, la **interactivitat** és clau, ja que permet a l'usuari explorar la xarxa activament per aclarir dubtes visuals.

G6. Heu considerat la interacció de l'usuari? Com afecta la comprensió de les dades? Sí, la interacció de l'usuari no només s'ha considerat, sinó que la identifiquem com un **requisit fonamental** per a l'exploració i comprensió efectiva

en 3D. Les accions bàsiques com rotar, fer zoom o desplaçar la càmera permeten a l'usuari:

- **Resoldre l'oclusió:** Canviant el punt de vista, l'usuari pot veure parts de la xarxa que estaven ocultes.
- **Construir un model mental:** L'exploració activa des de múltiples perspectives ajuda a formar una comprensió més robusta de l'estructura tridimensional de la xarxa.
- **Enfocar-se en detalls:** El zoom permet inspeccionar regions d'interès específiques.

Aquestes interaccions modifiquen les matrius de transformació (càmera i projecció), i la GPU re-executa eficientment el pipeline per generar noves vistes en temps real, la qual cosa fa que l'exploració sigui fluida.

G7. Com podríeu escalar el vostre sistema per xarxes més grans (milers de nodes)? L'escalabilitat és un repte clau dels mètodes bàsics, que tenen un cost de $O(|V|^2)$. Per a xarxes més grans, proposem una estratègia combinada:

- **Mètodes FDL Multi-Nivell:** Són la solució principal per escalar el posicionament dels nodes. Aquests mètodes redueixen la complexitat del graf (coarsening), calculen un layout global sobre la versió simplificada i després el refinem a l'escala original (uncoarsening).
- **Edge Bundling accelerat per GPU:** Per gestionar les arestes, utilitzaríem tècniques altament paral·lelitzables com el **KDE-Based Bundling** o les **variants geomètriques**, que són molt més ràpides que el FDEB. Implementar aquests mètodes a la GPU permetria processar milions d'arestes en temps real.

La combinació d'un FDL multi-nivell amb un EB basat en GPU és la via més prometedora per a la visualització de xarxes massives.

G8. Quines eines o llibreries heu utilitzat i per què? En el nostre estudi hem identificat i considerat diverses llibreries estàndard per a la implementació d'aquests sistemes:

- **JavaScript/WebGL:** Llibreries com **3d-force-graph**, que es basen en **Three.js** i motors de forces com **d3-force-3d**. Les hem escollit per la seva facilitat per crear visualitzacions de xarxes 3D interactives directament al navegador, la qual cosa facilita enormement la difusió i l'ús.
- **Python:** Per a la part de càlcul del layout i prototipatge, hem utilitzat **NetworkX** (concretament, la funció `spring_layout(dim=3)`). Per a la visualització dels resultats obtinguts amb Python, hem considerat llibreries com **Plotly**, **Matplotlib** o **Mayavi**. Aquestes eines són potents i formen part de l'ecosistema estàndard de ciència de dades a Python.

G9. Com heu validat la qualitat dels layouts? La qualitat d'un layout és un concepte multifactorial, que inclou aspectes quantitatius i qualitius. L'hem validat segons un conjunt de criteris establerts en la literatura:

- **Criteris estètics i de llegibilitat:** Hem avaluat visualment si els layouts aconseguien una distribució uniforme dels nodes, revelaven simetries o clústers de manera clara i minimitzaven la congestió.
- **Fidelitat estructural:** En algorismes com Kamada-Kawai, la qualitat es mesura per com de bé la distància geomètrica en el dibuix es correspon amb la distància teòrica en el graf.
- **Reducció del desordre:** Per a l'Edge Bundling, un indicador clau de qualitat és la seva capacitat per reduir l'efecte "hairball" i millorar la claredat de les rutes principals.

És important notar que la "qualitat visual" té un component subjectiu i depèn tant de la implementació específica com de les dades d'entrada.

G10. Si haguéssiu de millorar el vostre projecte, què canviariéu o afegiríeu? Basant-nos en l'estat de l'art i els reptes identificats, les futures millores se centrarien a portar el projecte més enllà de la visualització interactiva d'escriptori:

- (a) **Suport per a grafs dinàmics:** Adaptariem els algorismes per visualitzar xarxes que canvien amb el temps, mantenint l'estabilitat visual entre estats consecutius.
- (b) **Entorns immersius (VR/AR):** L'extensió més natural seria adaptar la visualització a entorns de realitat virtual. La percepció de profunditat i la navegació intuïtiva en un espai 3D immersiu podrien resoldre molts dels problemes de les pantalles 2D.
- (c) **Implementació d'algorismes avançats:** Implementariem un algorisme FDL multi-nivell complet i una tècnica d'Edge Bundling accelerada per GPU (com KDE) per millorar dràsticament l'escalabilitat i el rendiment en temps real per a xarxes realment massives.

Preguntes específiques sobre Force-Directed Layout (FDL)

Fonaments teòrics

F1. Explica què és un Force-Directed Layout. Quin és el seu principi bàsic?

Un Force-Directed Layout (FDL) és una família de mètodes per dibuixar grafs que es basa en una **analogia amb un sistema físic**. El seu principi bàsic és simular aquest sistema on els **nodes es consideren objectes que es repel·leixen** mútuament (com partícules amb la mateixa càrrega elèctrica), i les **arestes actuen com a molles que atrauen** els nodes que connecten. L'algorisme busca de manera iterativa una configuració de les posicions dels nodes que minimitzi l'energia total del sistema, arribant a un estat d'equilibri on les forces es balancegen.

F2. Quines són les forces que s'utilitzen en un FDL? Com es defineixen? Les dues forces fonamentals que s'utilitzen són:

- **Forces d'atracció (f_a):** Actuen entre nodes connectats per una aresta i tendeixen a apropar-los. S'apliquen al llarg de la línia que uneix els dos nodes.

- **Forces de repulsió (f_r):** Actuen entre tots els parells de nodes (o com a mínim, els no adjacents) i tendeixen a separar-los per evitar que se solapin i per distribuir-los per l'espai.

Les fórmules matemàtiques que defineixen aquestes forces varien segons l'algorisme. Per exemple:

- **Eades (1984):** Utilitza forces logarítmiques per a l'atracció i la llei de la inversa per a la repulsió:

$$f_a(d) = c_1 \log(d/c_2) \quad , \quad f_r(d) = c_3/d$$

- **Fruchterman & Reingold (1991):** Utilitza forces basades en una distància òptima k :

$$f_a(d) = d^2/k \quad , \quad f_r(d) = k^2/d$$

F3. Per què cal minimitzar una funció d'energia i què representa aquesta energia? L'objectiu de minimitzar una funció d'energia $E(L)$ és trobar la disposició (L^*) del graf que sigui **òptima segons certs criteris estètics** predefinitos. La funció d'energia està dissenyada de manera que les seves configuracions de baixa energia corresponguin a dibuixos "bons": aquells on els nodes estan ben distribuïts, les arestes tenen longituds uniformes i hi ha pocs encreuaments.

Aquesta energia representa una mesura de la "tensió" o "estrès" del sistema. Per exemple, en l'algorisme de Kamada-Kawai, l'energia mesura la discrepància entre les distàncies ideals (del graf) i les distàncies reals (en el dibuix). Un estat de mínima energia és un estat d'equilibri on les forces netes són properes a zero, resultant en un layout estable i, idealment, estèticament agradable i informatiu.

Algorismes concrets

A1. Com funciona l'algorisme de Eades? Quines són les seves fórmules de força i limitacions?

- **Funcionament:** Modela el graf com un sistema mecànic de molles i anelles d'acer, on les forces de les molles mouen el sistema fins a assolir un estat d'energia mínima.
- **Fórmules de força:** Per evitar que les molles siguin massa fortes a llargues distàncies (com passaria amb la llei de Hooke), utilitza forces logarítmiques per a l'atracció i una força inversament proporcional a la distància per a la repulsió:

$$f_a(d) = c_1 \log(d/c_2) \quad , \quad f_r(d) = c_3/d$$

- **Limitacions:**
 - El càlcul de la força repulsiva entre tots els parells de nodes té un cost computacional elevat, $\Theta(|V|^2)$ per iteració.
 - Pot tenir una convergència lenta i quedar atrapat en mínims locals, produint layouts subòptims.
 - La qualitat del resultat final és molt sensible a l'elecció inicial de les constants c_1, c_2, c_3 .

A2. Què introdueix l'algorisme de Fruchterman-Reingold respecte a Eades?

L'algorisme de Fruchterman-Reingold (FR) perfecciona el model físic d'Eades, principalment amb la introducció del concepte de "**temperatura**" per guiar la convergència del sistema, però també utilitzant un model de forces diferent i més intuïtiu.

El seu funcionament es basa en un procés iteratiu:

- **Definició de forces:** En lloc de forces logarítmiques, FR utilitza forces polinòmiques que depenen d'una "distància òptima" k entre nodes. Aquesta distància k es calcula en funció de l'àrea de dibuix i el nombre de vèrtexs.
 - La **força d'atracció** entre dos nodes connectats és $f_a(d) = \frac{d^2}{k}$
 - La **força de repulsió** entre qualsevol parell de nodes és $f_r(d) = \frac{k^2}{d}$
- **Càlcul de desplaçaments:** A cada iteració, es calculen les forces de repulsió entre tots els nodes i les forces d'atracció entre els nodes veïns. La suma de totes aquestes forces dóna un vector de desplaçament per a cada node.
- **Control per temperatura:** Aquí rau la seva principal innovació. El desplaçament de cada node no s'aplica directament, sinó que es limita per un valor de temperatura global, t . Aquesta temperatura comença sent alta, la qual cosa permet grans moviments que ajuden el layout a assolir una bona configuració global i a **evitar mínims locals** poc profunds.
- **Refredament simulat:** Després de cada iteració, la temperatura es redueix lleugerament ("refredament"). Això fa que els ajustaments al layout siguin cada cop més fins, estabilitzant el sistema gradualment en una configuració de baixa energia.

Aquest mecanisme de refredament millora notablement la qualitat i la convergència, produint generalment layouts estèticament agradables i amb una bona distribució dels nodes.

A3. Què aporta l'algorisme de Tutte? Per a quins grafs és aplicable?

L'algorisme de Tutte és un mètode històricament fonamental que, en lloc de simular forces explícites de repulsió, utilitza un enfocament matemàtic per aconseguir un dibuix amb propietats molt fortes.

- **Aportació principal:** La seva gran contribució és que garanteix matemàticament la generació d'un **dibuix planar (sense encreuaments d'arestes)** on totes les cares són polígons convexos.
- **Funcionament:** L'algorisme es basa a resoldre un sistema d'equacions lineals que sorgeix de la condició baricèntrica". El procés consta de tres passos clau:
 - (a) **Fixació de la cara externa:** Primer, se selecciona una cara del graf (normalment, la cara externa) i els seus vèrtexs es col·loquen en posicions fixes que formen un polígon convex (per exemple, sobre una circumferència). Aquestes posicions són les úniques que es defineixen manualment.
 - (b) **Plantejament del sistema d'equacions:** Per a cada vèrtex intern lliure v , s'estableix la condició que la seva posició (x_v, y_v) ha de ser el **baricentre** (la mitjana aritmètica) de les posicions dels seus veïns. Això

es tradueix en dues equacions lineals per a cada vèrtex lliure, una per a cada coordenada:

$$x_v = \frac{1}{\text{grau}(v)} \sum_{(u,v) \in E} x_u \quad \text{i} \quad y_v = \frac{1}{\text{grau}(v)} \sum_{(u,v) \in E} y_u$$

Com que les posicions dels veïns de la cara externa són conegudes, es forma un gran sistema d'equacions lineals on les incògnites són les coordenades dels vèrtexs lliures.

(c) **Resolució:** Aquest sistema té una solució única que es pot trobar amb mètodes estàndard, com l'eliminació gaussiana o mètodes iteratius. La solució d'aquest sistema proporciona les coordenades exactes per a tots els vèrtexs interns, resultant en el dibuix planar garantit.

- **Aplicabilitat:** Les seves fortes garanties teòriques, però, estan restringides a una classe específica de grafs: els **grafs planars 3-connectats**. La seva generalització a grafs no planars o a l'espai 3D no és directa i perd les propietats que el fan tan especial.

A4. Com funciona l'algorisme de Kamada-Kawai i com es diferencia dels anteriors?

- **Funcionament:** L'algorisme de Kamada-Kawai (KK) busca que la **distància geomètrica euclidiana** entre cada parell de nodes en el dibuix sigui el més proporcional possible a la seva **distància teòrica** (la longitud del camí més curt entre ells en el graf). Per fer-ho, minimitza una funció d'energia global. En cada pas, identifica el vèrtex amb el desequilibri de forces més gran i el mou per reduir l'energia localment, utilitzant el mètode de Newton-Raphson.
- **Diferències clau:**
 - (a) **Abast global vs. local:** A diferència d'Eades i FR, que es basen en forces locals, KK es basa en una propietat global (distàncies de camí més curt entre tots els parells de nodes). Això fa que reflecteixi millor l'estructura global i les simetries del graf.
 - (b) **Actualització per un vèrtex:** En cada iteració principal, KK mou només un vèrtex, el que té el gradient d'energia més alt, mentre que Eades i FR calculen desplaçaments per a tots els vèrtexs simultàniament.

A5. Per què l'algorisme de KK no és adequat per grafs molt grans? La principal raó és el **cost computacional extremadament alt de la seva fase d'inicialització**. L'algorisme requereix calcular les distàncies de camí més curt entre tots els parells de vèrtexs del graf. Aquest pas té una complexitat computacional molt elevada:

- Utilitzant l'algorisme de Floyd-Warshall, el cost és de $\Theta(|V|^3)$.
- Utilitzant l'algorisme de Dijkstra des de cada vèrtex, el cost és de $\Theta(|V| \cdot (|E| + |V| \log |V|))$.

Aquesta complexitat fa que el pre-càlcul sigui inviable per a grafs grans.

Integració gràfica

I1. Com s'integra el layout FDL en el pipeline gràfic 3D? El càlcul del layout FDL actua com una **etapa de preprocessament que s'executa a la CPU**, abans que comenci el renderitzat en temps real. El procés és el següent:

- (a) L'algorisme FDL pren com a entrada la definició del graf ($G = (V, E)$).
- (b) De manera iterativa, calcula les forces i actualitza les posicions fins a obtenir un conjunt final de coordenades 3D $\{p_v^*\}$ per a cada node.
- (c) Aquestes coordenades defineixen les primitives geomètriques (esferes, cilindres, etc.) a l'**Espai de Món (World Space)**.
- (d) Aquesta geometria ja posicionada és la que **serveix d'entrada al pipeline gràfic de la GPU** per a la seva visualització final.

I2. Quines són les etapes principals del pipeline de visualització (GPU)? Un cop la geometria 3D està definida, les etapes principals del pipeline a la GPU, tal com s'ha vist a l'assignatura, són:

- (a) **Transformacions de Modelatge i Vista:** Es posicionen els objectes i la càmera a l'espai de món i després es transforma tot a l'espai de càmera.
- (b) **Il·luminació (Shading):** Es calcula el color base dels vèrtexs o fragments segons llums i materials.
- (c) **Projecció i Clipping:** Els vèrtexs es projecten a l'espai de clip i es retalla la geometria fora del volum de visió.
- (d) **Rasterització:** Les primitives 3D es converteixen en un conjunt de fragments 2D.
- (e) **Test de Visibilitat:** S'utilitza el Z-Buffer per resoldre l'oclusió i determinar quin fragment és visible a cada píxel.

I3. Quina és la funció dels shaders (vertex/fragment) en aquest context? En un pipeline modern i programable, els shaders són essencials per controlar el procés de renderitzat:

- El **Vertex Shader** s'executa un cop per cada vèrtex. La seva funció principal és realitzar les **transformacions geomètriques** (Model-View i Projecció) per calcular la posició final del vèrtex (`gl_Position`). També pot realitzar càlculs d'il·luminació per vèrtex (Gouraud shading) i passar el color resultant al fragment shader.
- El **Fragment Shader** s'executa un cop per cada fragment generat durant la rasterització. La seva funció és **determinar el color final** d'aquest fragment. Aquí és on es fan els càlculs d'il·luminació per píxel (com Phong shading, que ofereix més qualitat), s'apliquen textures i es poden implementar efectes visuals avançats.

Implementació i rendiment

R1. Quin és el cost computacional dels mètodes FDL bàsics i com es pot reduir?

- **Cost computacional:** El principal coll d'ampolla dels mètodes FDL bàsics (com Eades o Fruchterman-Reingold) és el càlcul de les forces de repulsió entre cada parell de nodes. Això resulta en una complexitat per iteració de $\Theta(|V|^2)$, la qual cosa els fa poc pràctics per a grafs grans.
- **Com reduir-lo:**
 - (a) **Mètodes Multi-Nivell:** És l'estratègia més efectiva. En lloc de treballar sempre amb el graf complet, es crea una jerarquia de grafs més senzills per accelerar la convergència i el càlcul.
 - (b) **Tècniques d'aproximació:** Es pot evitar el càlcul de $O(|V|^2)$ forces utilitzant estructures de dades espacials (com octrees en 3D). Aquestes estructures permeten agrupar nodes llunyans i aproximar la seva força, reduint la complexitat a $\Theta(|V| \log |V|)$ o $\Theta(|V|)$.

R2. Com funcionen els mètodes multi-nivell i quins avantatges tenen?

- **Funcionament:** Els mètodes multi-nivell aborden l'escalabilitat mitjançant un procés de tres etapes:
 - (a) **Coarsening (Agregació):** El graf original i gran (G_0) es redueix iterativament per crear una jerarquia de grafs cada cop més petits i senzills ($G_1, \dots, G_m$).
 - (b) **Layout del graf més simple:** Es calcula un layout per al graf més petit (G_m) utilitzant un FDL estàndard.
 - (c) **Uncoarsening (Desagregació) i Refinement:** El layout es propaga cap amunt a través de la jerarquia. La disposició del graf G_{i+1} s'utilitza com a posició inicial per al graf més detallat G_i , i s'apliquen unes poques iteracions de FDL per refinar localment les posicions.
- **Avantatges:**
 - **Escalabilitat:** Milloren significativament el rendiment, permetent dibuixar grafs molt grans.
 - **Qualitat global:** Sovint produeixen layouts de millor qualitat, ja que la fase inicial captura l'estructura global i ajuda a evitar mínims locals profunds.

R3. Quina diferència hi ha entre coarsening i uncoarsening? Són les dues fases principals i oposades del paradigma multi-nivell:

- **Coarsening (agregació):** És el procés de **simplificació** del graf. Es parteix del graf original i es va reduint la seva mida iterativament per crear una jerarquia de grafs més petits i menys detallats.
- **Uncoarsening (desagregació):** És el procés de **refinement** i expansió. Es parteix del layout del graf més simple i es va propagant cap amunt en la jerarquia. A cada pas, s'utilitza el layout del nivell anterior com a una bona aproximació inicial per al nivell actual, que es refina amb unes poques iteracions de FDL.

R4. Quins reptes afegeix la representació 3D al FDL? Encara que els algorismes FDL s'estenen de manera natural al 3D, la representació i percepció de la visualització resultant en 3D introdueix reptes significatius que no existeixen o són menors en 2D:

- **Oclusió:** És el problema més gran. Nodes i arestes poden quedar ocults darrere d'altres.
- **Distorsió de perspectiva:** Fa que la percepció de la mida i la distància sigui dependent de la profunditat.
- **Navegació complexa:** L'exploració d'un espai 3D és inherentment més difícil.
- **Llegibilitat del text:** Posicionar etiquetes de manera llegible és complicat.

Aquests reptes fan que la **interactivitat** sigui absolutament essencial per a una exploració efectiva.

Aplicacions i consideracions

C1. En quins contextos reals pot ser útil el layout FDL en 3D? El layout FDL en 3D és útil en qualsevol domini on s'analitzen xarxes complexes i les relacions espacials poden revelar patrons d'interès. Alguns contextos reals inclouen:

- **Xarxes Socials:** Per identificar comunitats, actors influents i la seva interacció.
- **Biomedicina:** Per visualitzar xarxes de proteïnes, xarxes genètiques o connectomes cerebrals.
- **Enginyeria de Software:** Per analitzar l'arquitectura de sistemes de programari, dependències entre mòduls o grafs de crides.
- **Disseny de circuits (VLSI):** Per a la disposició de components i connexions en xips 3D.

C2. Com contribueix la visualització FDL a entendre la prominència o la cohesió en xarxes socials? La visualització FDL fa que conceptes analítics abstractes com la prominència i la cohesió siguin **visualment evidents** gràcies a la disposició espacial resultant:

- **Cohesió:** Els grups cohesius (comunitats o clústers) són conjunts de nodes densament connectats entre si. En un FDL, les fortes forces d'atracció internes a aquests grups els mantenen junts, mentre que les forces de repulsió els separen d'altres grups. El resultat és que les comunitats **apareixen visualment com a cúmuls densos de nodes**, fent la seva identificació immediata.
- **Prominència:** La prominència o centralitat d'un actor es refereix a la seva importància estructural. En un layout FDL, els nodes amb alta centralitat són atrets per molts altres nodes. Això sovint els situa en **posicions centralitzades dins del seu clúster o del graf sencer**, fent-los visualment destacats i fàcils d'identificar.

C3. Has tingut problemes amb mínims locals? Com ho gestiona l'algorisme? Sí, els mínims locals són un problema inherent i ben conegut dels algorismes FDL. Un mínim local és una configuració estable però subòptima. En el nostre estudi, hem vist que els algorismes moderns incorporen mecanismes específics per gestionar aquest problema:

- (a) **Refredament Simulat (Temperatura):** Aquesta tècnica, introduïda per Fruchterman-Reingold, és la principal defensa contra els mínims locals. Començant amb una "temperatura" alta, l'algorisme permet grans desplaçaments, la qual cosa ajuda a "saltar" fora de mínims locals poc profunds. A mesura que la temperatura baixa, el sistema s'estabilitza en una configuració de baixa energia.
- (b) **Mètodes Multi-Nivell:** Són una altra solució molt potent. En calcular primer un layout per a una versió molt simplificada del graf, estableixen una bona disposició global. Aquesta disposició guia el posicionament en els nivells més detallats, evitant que l'algorisme quedi atrapat en un mal mínim local.